Session 20 — Study Guide: Prisma Transactions, Atomic Issue Keys

Status: FINAL (reconciled 2026-10-27)

This guide is written for someone who sat through the session but is reviewing it alone tonight, maybe for the first time really thinking hard about Prisma transactions and Prisma migrations. If a sentence below assumes you remember something from Session 18 or 19, it says so and reminds you what that something was — you should never have to go dig through an old PDF to follow this one.

What we learned today

Three real things, in this order: (1) **why** generating a human-readable issue key like `WEB-3` is genuinely dangerous without extra protection; (2) **how** Prisma's interactive `$transaction` fixes that, using the exact same real `createIssueWithKey()` method you saw live; (3) **how** to prove, for real, that the fix actually works — both when something fails partway through (rollback) and when two people hit the same button at almost the same instant (concurrency). Your team also self-learned a new kind of lookup (`findUnique` by a compound key) during teamwork.

Why we needed it

Every issue in a project needs a short, human-friendly key — `WEB-1` , `WEB-2` , `WEB-3` — instead of a raw database id like `47` . Humans remember and say "WEB-3" out loud; nobody says "issue 47." That number has to come from somewhere, and it has to never repeat inside the same project. That "never repeat, always next" requirement is the entire reason today's session exists.

Mental model

Think of each project's key counter (`issueSeq`) as a **physical ticket dispenser** at a bakery. Only one person can tear off a given ticket number — the dispenser mechanism itself prevents two customers from ever holding ticket `"42"` at the same time. A database transaction is what turns `issueSeq` into that kind of dispenser: it guarantees "take the next number" and "use that number" happen together, as one indivisible action, with no other customer able to sneak in between them.

The database problem first

Without any protection, generating `WEB-3` would look like two separate steps in your code:

TEXT

```
1. Read Project.issueSeq            (currently 2)
2. Compute the next number          (2 + 1 = 3)
3. Create the Issue with key WEB-3
4. Save Project.issueSeq = 3
```

Two real, different things can go wrong here, and you need to be able to name both:

- **A partial failure.** Step 4 never runs (the app crashes, the request times out) — `issueSeq` still says `2`, but an Issue with key `WEB-3` already exists. The counter and the data have silently disagreed.
- **A race.** Two requests both read `issueSeq = 2` before either one writes anything back. Both compute `3`. Both try to create `WEB-3`. One succeeds; the other is rejected — or worse, if nothing enforces uniqueness, both succeed and you now have two different issues both calling themselves `WEB-3`.

Block 4 (Session 18) already proved, by hand, with raw SQL, that the fix is a **transaction**: `BEGIN`, a real `UPDATE ... RETURNING` that reads-and-increments the counter in one atomic statement, an `INSERT` using that returned value, then `COMMIT` — or, if anything fails, a real `ROLLBACK` that undoes every statement in that unit, not just the last one. Today's entire session is that exact same idea, through Prisma, in TypeScript, instead of raw SQL at a `psql` prompt.

First-Time Tool Box — Prisma interactive transactions and multi-step migrations

Session 19 only ever used Prisma for single, independent calls (`prisma.user.findMany()`, one `migrate dev` run on an empty database). Today is genuinely the first time you use two Prisma features that behave differently from anything you've used so far.

What they are. An **interactive transaction** (`prisma.$transaction(async (tx) => {...})`) is a real callback where every database call made through `tx` belongs to one all-or-nothing unit. A **multi-step migration sequence** is more than one `migrate dev` / `migrate deploy` run, in order, each one a real, permanent, replayable file under `prisma/migrations/`.

Why we use them here. The transaction is what turns "read, then write" into one safe action (see above). The multi-step migration sequence exists because Prisma will not let you add a required column directly to a table that already has real rows in it (see "Common errors" below) — the safe path is always more than one small step.

Where/how you run them. The transaction is just TypeScript inside `api/src/prisma/` (`PrismaService`) — no separate command, it runs when your Nest server runs. Migrations run from `api/` with `npx prisma migrate dev --name <description>` (development) or `npx prisma migrate deploy` (applies already-written migration files, never prompts, never generates anything new).

The commands you actually need today:

BASH

```
npx prisma format                          # tidy schema.prisma formatting
npx prisma validate                         # schema.prisma is syntactically/relationally
valid
npx prisma migrate dev --name <name>       # generate AND apply a new migration from your
schema edit
npx prisma migrate deploy                   # apply an already-written migration file, never
prompts
npx prisma generate                         # regenerate the typed Prisma Client after a
schema change
```

How to tell it worked. `migrate dev` / `migrate deploy` print `Your database is now in sync with your schema.` `validate` prints `The schema at prisma/schema.prisma is valid`. If your Nest server was already running, restart it after `prisma generate` — an already-running process keeps using the OLD generated client in memory and will not see new fields.

Common first-time mistakes with these two tools: forgetting to restart the Nest server after `prisma generate` (you'll get confusing "field does not exist" errors even though the schema is correct); and reading only Prisma's one-line summary instead of opening the real generated `.sql` file to see exactly what it's about to run.

The solution, step by step

Step 1 — the schema gets a real place to put the key

PRISMA

```
model Issue {
  id         Int      @id @default(autoincrement())
  projectId  Int      @map("project_id")
  key        String
  title      String
  status     String   @default("todo")
  ...
  @@unique([projectId, key])
}
```

- `key String` — a plain text field. Nothing in the database generates `WEB-3` automatically; that's application logic, built in Step 3 below, on purpose, by hand.
- `@@unique([projectId, key])` — a **compound** unique constraint: the PAIR `(projectId, key)` must be unique, not `key` alone. This is why `WEB-1` and `API-1` can both exist — they belong to different projects — but two `WEB-1`s in the SAME project is a real, structural impossibility, not just a convention.

Step 2 — getting there safely when a table already has real rows

Prisma genuinely refuses to add a required column directly to a table that already has data, without you deciding what existing rows should contain:

TEXT

```
⚠  We found changes that cannot be executed:
  • Step 0 Added the required column `key` to the `issues` table without a default value.
    There are 7 rows in this table, it is not possible to execute this step.
```

This is not a confirmable warning — Prisma has no value to put in `key` for rows that already exist, so it stops. The safe fix is always the same shape, in three real, versioned migrations, never one raw hand-typed SQL command run outside the migration history:

1. **Add the column as optional first** (`key String?`, plus the project's own counter `issueSeq Int @default(0)`). An optional, defaulted column needs no decision about existing rows, so this step always succeeds cleanly.

2. **Backfill existing rows with a real migration file** — not a one-off `psql` command. This exact SQL (real, verified, re-run successfully on two independent databases) assigns each existing issue a correct, sequential key per project:

TEXT

```
UPDATE issues i
SET key = p.key || '-' || sub.rn
FROM (
  SELECT id, project_id,
         ROW_NUMBER() OVER (PARTITION BY project_id ORDER BY id) AS rn
  FROM issues
) sub
JOIN projects p ON p.id = sub.project_id
WHERE i.id = sub.id;
```

Read `ROW_NUMBER() OVER (PARTITION BY project_id ORDER BY id)` as: "number the rows 1, 2, 3... separately within each project, in id order" — exactly the numbering a human would do by hand, done once, correctly, for every row.

1. **Only now tighten the constraint** — `key` becomes required, `@@unique([projectId, key])` gets added. By this point every row already has a real key, so this step is safe.

Use this state-based rule to decide which path applies to a given database, not a headcount of who finished what: **if your own `issues` table has NO existing rows at all**, there is nothing to decide for — the final shape can be written directly in one plain migration. **If your own `issues` table already has real rows**, the three-step dance above is the safe path — exactly the situation a production team hits constantly, and a genuinely reusable pattern, not a one-off trick for this session.

Step 3 — the real transaction that generates the key

TS

```
async createIssueWithKey(projectId: number, title: string) {
  return this.$transaction(async (tx) => {
    const project = await tx.project.update({
      where: { id: projectId },
      data: { issueSeq: { increment: 1 } },
    });

    return tx.issue.create({
      data: {
        projectId,
        key: `${project.key}-${project.issueSeq}`,
        title,
        status: 'todo',
      },
    });
  });
}
```

Read it in three small pieces:

Piece A — opening the transaction.

TS

```ts
return this.$transaction(async (tx) => {
```

What is it? A real callback, given its own transaction-scoped client, `tx`. Why do we need it? Everything `await`ed through `tx` inside this callback becomes one all-or-nothing unit — no other request can see a half-finished result. What breaks without it? Exactly the partial-failure and race bugs described above.

Piece B — the atomic increment.

TS

```ts
const project = await tx.project.update({
  where: { id: projectId },
  data: { issueSeq: { increment: 1 } },
});
```

`where` picks the one project row. `{ increment: 1 }` is Prisma's own atomic column-increment operator — it compiles directly to a single SQL statement, `SET issue_seq = issue_seq + 1`, performed entirely inside Postgres. `project` is the UPDATED row Prisma hands back; `project.issueSeq` already holds the freshly-reserved number — not a value you computed yourself in JavaScript.

Piece C — using that fresh number immediately.

TS

```ts
return tx.issue.create({
  data: {
    projectId,
    key: `${project.key}-${project.issueSeq}`,
    title,
    status: 'todo',
  },
});
```

This is the part that batch `$transaction([op1, op2])` genuinely cannot do: `op2` in a batch array is built BEFORE anything runs, so it could never reference a value `op1` only produces once it actually executes. The interactive form you're reading can, because `project.issueSeq` is a real value already returned by the `await` just above it.

What to look at in your own project: `api/src/prisma/prisma.service.ts`, the `createIssueWithKey()` method. **What to expect when it works:** calling it twice in a row on the same project produces two different, correctly-sequential keys, and `issueSeq` goes up by exactly 2.

What happens under concurrency

"Concurrency" here means two separate calls to `createIssueWithKey()` that are BOTH in flight at the same time (fired with `Promise.all`, never one `await`ed before the other starts) — not two ordinary sequential requests that happen to arrive close together.

TEXT

```
Call A (project WEB, issueSeq currently 2)        Call B (SAME project WEB)
-------------------------------------------------  ----------------------------------------
----
UPDATE issues_seq = issue_seq + 1   -> 3           (tries the same UPDATE... must WAIT)
  [row is now locked by A's transaction]                      |
INSERT Issue key = WEB-3                                       | (still waiting)
COMMIT  [row lock released]                                    |
                                                  UPDATE issue_seq = issue_seq + 1   -> 4
                                                  INSERT Issue key = WEB-4
                                                  COMMIT
```

The key line to actually understand: **JavaScript is not creating this lock.** `Promise.all` only starts both calls at (roughly) the same moment — it has no idea what a database lock even is. PostgreSQL itself creates a real row-level lock the instant Call A's `UPDATE` statement touches that project's row, and holds it until Call A's transaction commits or rolls back. Call B's own `UPDATE` on that SAME row genuinely cannot proceed until the lock is released — that's what forces the two transactions to run one-after-another on that one row, with no explicit `SELECT ... FOR UPDATE` needed for this exact shape.

Two calls on TWO DIFFERENT projects never wait on each other at all — each transaction only locks the one project row it actually updates, so unrelated projects stay genuinely parallel.

What to look at: the live-coded concurrency script, two `createIssueWithKey()` calls started with `Promise.all` on the SAME project id. **What to expect:** two distinct, correctly-sequential keys, never a duplicate, and `issueSeq` advanced by exactly 2 — never lost, never double-counted.

What rollback means

Rollback is not a separate feature you turn on — it's the automatic result of a transaction failing partway through. The real proof shown live does this, in order:

TEXT

```
1. increment issueSeq        (inside the transaction)
2. create the Issue row      (inside the SAME transaction)
3. throw an error            (deliberately, AFTER both steps above already ran)
```

Because the thrown error happens inside the `$transaction` callback, PostgreSQL rolls BOTH earlier statements back together — not just the one that technically failed. What was actually observed when this ran for real:

TEXT

```
issueSeq before: 3   issueSeq after: 3     <- unchanged, the increment was undone
orphan issue row (should be null): null    <- the Issue was never really created
```

This is the exact guarantee this whole session exists to deliver: either the counter AND the issue both end up real, or neither does — never a world where one exists without the other.

Teamwork: looking an issue up by its own key

Live code only ever looks an issue up by its numeric `id` (inside the same transaction that creates it). Your team's own job was different: given a project and a human-readable key like `WEB-3`, find that one real issue — never shown live, entirely self-learned.

TS

```ts
findIssueByKey(
  @Param('id', ParseIntPipe) id: number,
  @Param('key') key: string,
) {
  return this.prisma.issue.findUnique({
    where: { projectId_key: { projectId: id, key } },
  });
}
```

What is it? `findUnique` given a COMPOUND key — `projectId_key` is Prisma's own generated input name, built by joining the two field names in `@@unique([projectId, key])`, in that exact order, with an underscore. **Why do we need it?** Neither `projectId` alone nor `key` alone is guaranteed unique — only the PAIR is (two different projects can each have their own `WEB-1` ... no, their own `-1`; the real point is two issues can share a `key` string across different projects, just never within the SAME project). **What breaks without it?** A plain filter object (`{ projectId, key }`) only works with `findFirst` / `findMany` — `findUnique` specifically requires a shape that matches a REAL unique constraint, which is exactly what `projectId_key` is. **What to look at:** `@@unique([projectId, key])` in `schema.prisma` — that declaration is what makes Prisma generate the `projectId_key` input name in the first place. **What to expect:** one real issue row back, or `null` if no issue in that project has that exact key.

Connecting to earlier sessions

- **Session 17** gave you the foreign-key relationships (`Project` → `Workspace`, `Issue` → `Project` / `User`) that today's `Issue` model is built on top of.
- **Session 18** is where you first wrote `BEGIN` / `UPDATE ... RETURNING` / `INSERT` / `COMMIT` by hand, in raw SQL, to solve this exact "generate a safe next number" problem. Everything in today's `createIssueWithKey()` is that same reasoning, now expressed through Prisma instead of raw SQL.
- **Session 19** is where Prisma itself, `schema.prisma`, and a single plain `migrate dev` run were introduced — but only ever against an empty table, and only ever for single, independent calls. Today is the first time either of those tools has to do something genuinely harder (see the First-Time Tool Box above).

Glossary

- **Interactive `$transaction`** — `prisma.$transaction(async (tx) => {...})`; a real callback whose every `await`ed call through `tx` belongs to one all-or-nothing database transaction.
- **Batch `$transaction`** — `prisma.$transaction([op1, op2])`; a plain array of operations, all built before any of them run — `op2` can never use a result `op1` only produces once it executes.
- **`{ increment: 1 }`** — Prisma's atomic column-increment operator; compiles to one SQL statement, `SET x = x + 1`, performed entirely inside the database, never read-then-computed in JavaScript.
- **Row-level lock** — a real lock PostgreSQL holds on one specific row from the moment an `UPDATE` touches it until that row's own transaction commits or rolls back.
- **Compound unique constraint** — `@@unique([a, b])`; the PAIR of values must be unique together, not either column alone.
- **`findUnique` with a compound input** — a lookup backed by a compound unique constraint, using Prisma's own generated field name (e.g. `projectId_key`, generated automatically from `@@unique([projectId, key])`); valid only for `findUnique`, not `findFirst` / `findMany`.
- **Migration** — a real, permanent, timestamped file under `prisma/migrations/`, recording one step of database structure or data change; `_prisma_migrations` is the real table that proves which ones have actually run.
- **`$queryRaw` / `$executeRaw`** — Prisma's tagged-template raw-SQL functions; a lower-level query capability usable inside either transaction style, not a third, competing transaction style.

Common beginner mistakes

- Reading `issueSeq` / `key` from a separate query made BEFORE the transaction, instead of from the transaction's own returned row — a value that could already be stale by the time you use it.
- Writing the increment as a plain read-then-write (`issueSeq: project.issueSeq + 1`) instead of `{ increment: 1 }` — this quietly reintroduces the exact race the transaction exists to prevent.
- Believing `Promise.all` alone, without Postgres's own row lock, is what prevents a duplicate key.
- Confusing `findUnique`'s compound input (`{ projectId_key: { projectId, key } }`) with a plain filter object (`{ projectId, key }`) — the plain object form is only valid for `findFirst`.

Common errors and how to debug them

- **"Added the required column `key` ... without a default value ... it is not possible to execute this step."** This means the table already has real rows and the migration you're trying to run gives Prisma no value to put in them. Inspect: does the table genuinely have rows (`SELECT COUNT(*) FROM issues;`)? If yes, you need the three-step safe sequence above, not a direct migration. Never run `prisma migrate reset` to make this message go away — that destroys the real data the error is correctly protecting.
- **`P2002` unique constraint violation on `issues_project_id_key_key`.** This means two attempts created the exact same `(projectId, key)` pair. If you ever see this outside a deliberate test, it means something generated a key OUTSIDE `createIssueWithKey()`'s own transaction — treat it as a real bug to find, not something to silently retry.

- **A schema field "doesn't exist" even though you just added it.** Inspect whether you ran `npx prisma generate` AND restarted your Nest server afterward — a server that was already running keeps the OLD generated client in memory.
- **A `$transaction` that looks "stuck."** Before assuming it's broken, check real wall-clock time against the actual number of concurrent callers — several transactions genuinely queued on the same row's lock is expected behavior, not a bug, for the shape of problem this session solves.

Review questions

1. What are the two real failure modes that appear without a transaction tying the increment and the insert together?
2. Why can't batch `$transaction([...])` solve today's problem?
3. What, specifically, makes two concurrent same-project operations never produce a duplicate key?
4. Why does adding `key` directly as a required column fail against a table that already has rows, and what three-step sequence fixes it?
5. Why are this session's two new routes not a "Mongoose→Prisma cutover"?

Answers

1. A partial-failure gap (the counter advances but no issue exists); a race (two concurrent operations compute the same next number and collide on the same key).
2. It builds the whole array of operations before any run — the second operation can never use the first one's own result.
3. Postgres's own row-level lock on the `UPDATE` statement, held until that transaction commits, forces the second transaction's own `UPDATE` to wait its turn.
4. Prisma has no value to put in the new required column for rows that already exist, so it refuses outright. The fix: add the column as optional first, backfill existing rows with a real migration file, then tighten the constraint only once every row already has a real value.
5. They are additive, scoped mechanism-proof routes under the existing `PrismaCheckModule` convention — the real, production `/issues` resource is still served entirely by the existing Mongoose-backed code, untouched.

What you should be able to explain now

Why the increment and the insert have to share one real transaction. Why interactive `$transaction` is the only one of Prisma's two transaction styles that fits this problem, and why raw SQL interop is a separate query capability, not a competing third style. How Postgres's own row lock — not anything written in JavaScript — is what actually prevents a duplicate key under real concurrency. Why adding a required column directly can fail against real data, and what three-step sequence makes it safe. Why nothing about this session put any existing Mongoose-backed route at risk.

Not yet

A real Mongoose→Prisma resource cutover for any production route; `Comments`; any further frozen `Issue` fields (`description` / `priority` / `storyPoints` / `sprintId` / `updatedAt`); any frontend framework; Prisma's own newer configuration-file/CLI workflow (this project remains deliberately pinned to the older, classic CLI version).